LAPORAN TUGAS BESAR
IF2224 Teori Bahasa Formal dan Otomata

"Milestone 1 - Lexical Analysis"

Dipersiapkan oleh:

Nama lengkap	**NIM**
Fauzan Mohamad Abdul Ghani	13524113
Safira Berlianti	13524128
Eduard Daniel Ariajaya	13524129
Muhammad Daffa Arrizki Yanma	13524133
Reysha Syafitri MR	13524137

Laboratorium Ilmu dan Rekayasa Komputasi
Program Studi Teknik Informatika
Sekolah Teknik Elektro dan Informatika
Institut Teknologi Bandung

Contents

1 Teori Singkat **2**
1.1 Pengertian, Fungsi, dan Peran Lexer 2
1.2 Kaitan Lexer dengan Token dan DFA 2

2 Perancangan dan Implementasi **3**
2.1 Perancangan Program 3
2.2 Hasil Implementasi 4
2.3 Diagram Transisi DFA 6

3 Pengujian **9**

4 Kesimpulan dan Saran **12**
4.1 Kesimpulan 12
4.2 Saran 13

5 Lampiran **13**

6 Referensi **15**

1 Teori Singkat

1.1 Pengertian, Fungsi, dan Peran Lexer

Analisis leksikal (*lexical analysis*) adalah tahap pertama dalam proses kompilasi atau interpretasi sebuah bahasa pemrograman. Komponen yang melakukan proses ini disebut sebagai *lexer* (atau *lexical analyzer*). Secara ringkas, pengertian, fungsi, dan peran *lexer* dalam proses kompilasi adalah sebagai berikut:

- **Pengertian:** *Lexer* adalah program yang bertugas membaca kode sumber (*source code*) mentah berupa rangkaian karakter dari kiri ke kanan.
- **Fungsi:** Fungsi utama *lexer* adalah mengelompokkan rangkaian karakter tersebut menjadi unit-unit bermakna dengan menggunakan pola tertentu. Selain itu, *lexer* juga berfungsi untuk membuang bagian kode yang tidak diperlukan untuk eksekusi, seperti spasi, *tab*, dan komentar. *Lexer* juga merupakan garis pertahanan pertama untuk mendeteksi kesalahan leksikal, seperti penggunaan simbol yang tidak dikenal.
- **Peran dalam Kompilasi:** Peran *lexer* sangat krusial untuk mempermudah tahap selanjutnya, yaitu analisis sintaks (*parsing*). Dengan adanya *lexer*, tahap *parsing* tidak perlu lagi berurusan dengan karakter mentah satu per satu, melainkan langsung bekerja dengan kumpulan unit terstruktur yang logis.

1.2 Kaitan Lexer dengan Token dan DFA

Dalam menjalankan tugasnya, *lexer* sangat erat kaitannya dengan konsep *Token* dan *Deterministic Finite Automata* (DFA):

- **Kaitan dengan Token:** Token adalah *output* yang dihasilkan oleh *lexer*. Token merupakan unit terkecil dalam program yang memiliki arti, seperti kata kunci (*keyword*), nama variabel (*identifier*), operator, angka, atau tanda baca. Ketika *lexer* menemukan urutan karakter yang cocok dengan suatu pola bahasa pemrograman, ia akan membungkusnya menjadi sebuah token yang terdiri dari jenis token (*type*) dan nilai token (*value*).
- **Kaitan dengan DFA:** Agar proses pengenalan pola karakter dan pembentukan token dapat dijalankan secara otomatis oleh komputer, aturan pembacaannya diimplementasikan ke dalam bentuk *Finite Automata*, khususnya *Deterministic Finite Automata* (DFA). *Lexer* beroperasi selayaknya mesin DFA; ia hanya boleh membaca satu karakter demi satu karakter dari input kode sumber, lalu memprosesnya (berpindah *state*) sesuai dengan aturan transisi

pada DFA hingga mencapai *accept state* yang menandakan sebuah token valid telah terbentuk.

2 Perancangan dan Implementasi

2.1 Perancangan Program

Perancangan program *lexer* yang telah dibuat adalah sebagai berikut:

- **Teknologi yang digunakan:** GNU C/C++ dan Makefile.
- **Struktur Program & Fungsi Utama:** Program dibangun dengan arsitektur *modular* berbasis *Object-Oriented Programming* (OOP) yang membagi tugas analisis leksikal ke dalam beberapa berkas sumber:
  - `main.cpp`: Merupakan titik masuk (*entry point*) program yang menangani argumen *command-line* untuk berkas input dan output, serta menginisialisasi kelas utama *lexer*.
  - `ArionLexer (.hpp/.cpp)`: Kelas pengontrol utama DFA (*Deterministic Finite Automaton*). Kelas ini mengatur pembacaan berkas, menentukan transisi *state* awal berdasarkan karakter yang dibaca, dan memanggil modul pembantu lainnya.
  - `KeywordIdentifierLexer (.hpp/.cpp)`: Modul khusus yang mengimplementasikan DFA untuk mengenali *keyword* (seperti `begin`, `end`, `var`) dan *identifier*. Modul ini menggunakan transisi *state* numerik secara eksplisit.
  - `OperatorDelimiterLexer (.hpp/.cpp)`: Modul khusus yang mengimplementasikan DFA untuk mengenali operator matematika/logika dan *delimiter* (seperti `:=`, `<=`, `;`).
  - `TipeDataKomentar (.hpp/.cpp)`: Berisi kelas-kelas *handler* untuk memproses literal angka (`NumberHandler`), literal *string*/karakter (`StringHandler`), serta komentar (`CommentHandler`).
- **Alur Kerja Program:** Program beroperasi dengan membaca berkas masukan `.txt` karakter demi karakter menggunakan `InputBuffer`. Pada kondisi `STATE_START` di `ArionLexer`, program mengevaluasi karakter saat ini. Jika berupa huruf, aliran dialihkan ke `KeywordIdentifierLexer`; jika berupa angka, dialihkan ke `NumberHandler`; jika berupa simbol, dialihkan ke `OperatorDelimiterLexer`, dan seterusnya. Modul yang bersangkutan akan terus membaca karakter hingga mencapai *accepting state* atau menemui karakter yang bukan bagian dari token tersebut. Setelah token utuh didapatkan,

program memetakan token ke nama token standarnya (melalui fungsi `normalizeToken`) lalu mencetaknya ke layar dan berkas keluaran.

- **Justifikasi Implementasi:** Pendekatan modular dipilih untuk menerapkan prinsip *Separation of Concerns*. Pembagian DFA besar menjadi DFA kecil yang spesifik (seperti pemisahan antara DFA *Identifier* dan DFA *Operator*) mencegah kode menjadi satu *switch-case* raksasa (*spaghetti code*). Penggunaan *state* numerik manual alih-alih *Regular Expression* (Regex) bawaan bahasa C++ dilakukan untuk mendemonstrasikan pemahaman mendasar mengenai teori otomata (DFA). Struktur kelas dan objek ini akan sangat mempermudah pemanggilan token secara *on-demand* saat membangun *Parser* di Milestone selanjutnya.

2.2 Hasil Implementasi

Berikut adalah dokumentasi dari komponen-komponen penting dalam implementasi *lexer*:

1. Komponen Pengontrol Utama (Main Loop DFA)

Bagian ini berada pada fungsi `ArionLexer::analyze()`. Fungsi ini mengimplementasikan *looping* utama yang membaca karakter dan menentukan cabang DFA mana yang harus dieksekusi.

Listing 1: Snippet loop utama ArionLexer::analyze

```
switch (state) {
    case STATE_START: {
        if (std::isspace((unsigned char)c)) {
            state = STATE_START; break;
        }
        if (isLetter(c)) {
            state = STATE_IDENT_KW;
            // ... alihkan ke KeywordIdentifierLexer
            break;
        }
        if (isDigit(c)) {
            state = STATE_NUMBER;
            emit(numHandler.process(inBuf, c));
            break;
        }
%          // ... penanganan karakter lain
%
```

Penjelasan: Snippet di atas menunjukkan bagaimana *state* awal mendistribusikan karakter masukan ke penangan (*handler*) yang tepat. Pengkondisian menggunakan fungsi seperti `isLetter()` dan `isDigit()` berfungsi sebagai transisi dari *start state* menuju *state* spesifik.

2. Komponen DFA Keyword dan Identifier

Bagian ini diimplementasikan dalam fungsi `KeywordIdentifierLexer::processChar(char c)`. Ini merupakan contoh penerapan DFA transisi eksplisit per karakter.

Listing 2: Snippet transisi state untuk keyword 'const'

```
if (state == 100) {
    buffer += c;
    if (lc == 'o') {
        state = 101; // "co"
        return true;
    } else if (lc == 'a') {
        state = 110; // "ca"
        return true;
    } else if (isLetter(c) || isDigit(c)) {
        state = 9999; // fallback ke identifier
        return true;
    }
    return false;
}
```

Penjelasan: Kode ini mencontohkan graf transisi secara harfiah. Ketika pembacaan diawali huruf 'c' (*state* 100), program mengecek karakter berikutnya. Jika 'o', ia bertransisi ke *state* 101 (jalur menuju *keyword* `const`). Jika huruf lain yang tidak membentuk *keyword*, alur transisi dilempar ke *state* 9999 yang merupakan *accepting state* universal untuk *Identifier*.

3. Komponen Input Buffer

Mekanisme ini sangat vital untuk menganalisis token ganda (seperti `:=` atau `<=`) tanpa kehilangan karakter jika ternyata karakter kedua tidak cocok.

Listing 3: Snippet class InputBuffer

```
char InputBuffer::get() {
    if (hasBuffer) {
```

```
        hasBuffer = false;
        return buffer;
    }
    return in.get();
}

void InputBuffer::put(char c) {
    hasBuffer = true;
    buffer = c;
}
```

Penjelasan: Fungsi `put(char c)` digunakan untuk mengembalikan karakter ke dalam sistem (menyimpannya di memori *buffer*) jika suatu DFA (misalnya DFA angka yang mengecek desimal) ternyata membaca karakter yang bukan ranahnya. Pembacaan selanjutnya melalui `get()` akan memprioritaskan isi *buffer* terlebih dahulu sebelum mengambil karakter baru dari berkas masukan asli.

2.3 Diagram Transisi DFA

Berikut adalah diagram transisi DFA (*Deterministic Finite Automaton*) dari *lexer* yang telah dirancang. Diagram ini memetakan aliran pembacaan karakter tunggal ke dalam status (*state*) tertentu hingga mencapai *accepting state* berupa token.

Penjelasan Alur *State* DFA:

Berdasarkan diagram di atas, DFA bermula pada *Start State* dan melakukan transisi dengan membaca karakter pertama dari aliran input. Alur tersebut terbagi menjadi beberapa cabang utama:

- **Jalur *Keyword* dan *Identifier* (Cabang Huruf):** Jika karakter pertama adalah huruf alfabet (`a-z, A-Z`), aliran masuk ke cabang pencarian *keyword*. Diagram menggunakan struktur yang mirip dengan graf *Trie*.
  - Contohnya: Saat membaca huruf 'p', *state* berpindah ke *q_p*. Jika karakter berikutnya adalah 'r', pindah ke *q_pr*, lalu 'o' ke *q_pro*, dan seterusnya hingga membentuk *keyword* seperti `program` atau `procedure`.
  - ***Fallback* ke *Identifier*:** Apabila di tengah pembacaan karakter terdapat huruf atau angka yang menyimpang dari rute *keyword* (misalnya setelah membaca 'pro' karakter selanjutnya adalah 'b', bukan 'c' atau 'g'), maka DFA secara otomatis keluar dari jalur *keyword* dan langsung masuk ke *accepting state q_ident*. *State* ini akan terus melakukan *looping* menerima huruf dan angka hingga bertemu spasi atau simbol.
- **Jalur Literal Numerik (Cabang Angka):** Jika input diawali dengan angka (`0-9`), DFA masuk ke state $q2$ dan akan terus membaca angka (*looping*).
  - Jika setelah kumpulan angka pembacaan langsung berhenti (bertemu spasi/simbol lain), DFA berhenti di *accepting state* `intcon()`.
  - Jika DFA menemukan karakter titik (`.`), ia beralih ke rute desimal. Jika setelah titik diikuti oleh angka, maka alur berakhir di *accepting state* `realcon()`.
- **Jalur Literal *String* dan Karakter (Cabang Tanda Kutip):** Diawali dengan karakter tanda kutip tunggal (`'`).
  - Jika di dalam kutip hanya ada satu karakter alfabet/simbol lalu langsung ditutup dengan kutip lagi (`'a'`), DFA berakhir di state `charcon()`.
  - Jika berisi lebih dari satu karakter atau berupa himpunan kata (`'word'`), DFA masuk ke state `string()`.
- **Jalur Operator Kata (*Word Operators*):** Sama seperti jalur *keyword*, operator yang berupa alfabet (seperti `div`, `mod`, `and`, `or`, `not`) memiliki cabang hurufnya masing-masing. Jika gagal membentuk susunan kata tersebut, ia akan kembali dilempar ke *state q_ident*.
- **Jalur Operator Simbol dan *Delimiter*:** DFA memiliki *state* pendek untuk simbol-simbol tunggal (seperti `+`, `-`, `*`, `/`, `;`, `,`, `(`, `)`, `[`, `]`).

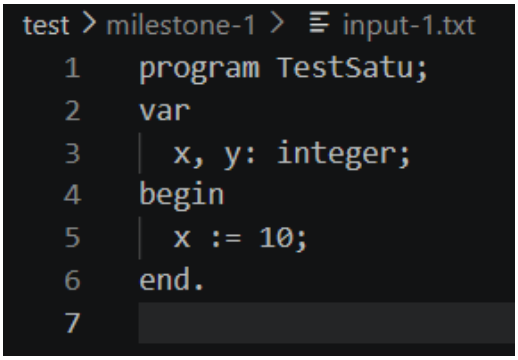
- **Jalur Komentar:** DFA mendukung dua jenis penanda komentar.
  - Jika membaca kurung kurawal buka ({), *state* akan terus membaca (mengabaikan) semua karakter apapun (*Any*) hingga menemukan kurung kurawal tutup (}), lalu masuk ke *state* **comment**.
  - Jika membaca karakter `(` yang diikuti langsung oleh `*`, *state* akan mengabaikan seluruh karakter hingga bertemu pola penutup `*` yang diikuti dengan `)`, untuk masuk ke *state* **comment**.

3 Pengujian

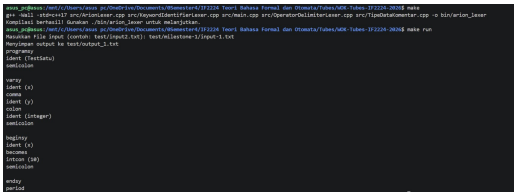
Berikut adalah dokumentasi 10 pengujian mandiri dengan kasus yang unik untuk memastikan kebenaran pembacaan token. Setiap kasus uji menyertakan bukti tangkapan layar (screenshot) masukan (input) program dan keluaran (output) daftar token yang dihasilkan.

1. **Kasus Uji 1: Deklarasi Program & Variabel Dasar**
   Tujuan: Menguji pembacaan struktur dasar program, deklarasi variabel, dan assignment sederhana.

Input:



Output:



2. **Kasus Uji 2: Tipe Data String dan Karakter**
   Tujuan: Menguji pengenalan token string dan char yang diapit oleh tanda petik tunggal di dalam blok eksekusi.

Input:

```
test > milestone-1 > input-2.txt
1   begin
2     v1 := 'A';
3     v2 := 'Tubes TBFO';
4   end.
```

Output:

```
asus_pc@asus:/mnt/c/Users/asus pc/OneDrive/Documents/05semester4/IF2224 Teori Bahasa Formal dan Otomata/Tubes/KOK-Tubes-IF2224-2026$ make run
Masukkan file input (contoh: test/input2.txt): test/milestone-1/input-2.txt
Menyimpan output ke test/output_2.txt
beginsy
ident (v1)
becomes
charcon ('A')
semicolon

ident (v2)
becomes
string ('Tubes TBFO')
semicolon

endsy
perlad
```

3. **Kasus Uji 3: Operasi Aritmatika Kompleks (Integer)**
   Tujuan: Menguji pembacaan ekspresi matematika dasar beserta prioritas tanda kurung.

Input:

```
test > milestone-1 > input-3.txt
1   begin
2     hasil := (10 + 5) * 2 div 3;
3   end.
```

Output:

```
Masukkan file input (contoh: test/input2.txt): test/milestone-1/input-3.txt
Menyimpan output ke test/output_3.txt
beginsy
ident (hasil)
becomes
lparent
intcon (10)
plus
intcon (5)
rparent
times
intcon (2)
idiv
intcon (3)
semicolon

endsy
perlad
```

4. **Kasus Uji 4: Bilangan Riil, Pembagian Riil, dan Modulo**
   Tujuan: Menguji transisi DFA dari integer ke real (pembacaan tanda titik) serta operator pembagian riil dan modulo.

Input:

```
test > milestone-1 > input-4.txt
1   begin
2     luas := 3.14 / 2.0;
3     sisa := 10 MOD 3;
4   end.
```

Output:

```
asus_pc@asus:/mnt/c/Users/asus pc/OneDrive/Documents/05semester4/IF2224 Teori Bahasa Formal dan Otomata/Tubes/KOK-Tubes-IF2224-2026$ make run
Masukkan file input (contoh: test/input2.txt): test/milestone-1/input-4.txt
Menyimpan output ke test/output_4.txt
beginsy
ident (luas)
becomes
realcon (3.14)
rdiv
realcon (2.0)
semicolon

ident (sisa)
becomes
intcon (10)
imod
intcon (3)
semicolon

endsy
perlad
```

5. **Kasus Uji 5: Operasi Relasional dan Logika Bersarang**
   Tujuan: Menguji operator boolean, pembanding persamaan, dan negasi (NOT) dalam blok kondisional if-then.

Input:

```
test > milestone-1 > input-5.txt
1  begin
2    if (x >= 10) AND NOT (y == 5) then
3      x := 0;
4  end.
```

Output:

```
asus_pc@asus:/mnt/c/Users/asus pc/OneDrive/Documents/05semester4/IF2224 Teori Bahasa Formal dan Otomata/Tubes/MDK-Tubes-IF2224-2026$ make run
Masukkan File input (contoh: test/input2.txt): test/milestone-1/input-5.txt
Menyimpan output ke test/output_5.txt
beginsy
ifsy
lparent
ident (x)
geq
intcon (10)
rparent
andsy
notsy
lparent
ident (y)
eql
intcon (5)
rparent
thensy
ident (x)
becomes
intcon (0)
semicolon

endsy
period
```

6. **Kasus Uji 6: Variasi Format Komentar**
   Tujuan: Menguji kemampuan lexer dalam mendeteksi dan menghasilkan token khusus komentar untuk dua format berbeda yaitu { } dan (* *).

Input:

```
test > milestone-1 > input-6.txt
1  begin
2    { Komentar format pertama }
3    x := 1;
4    (* Komentar format kedua *)
5  end.
```

Output:

```
asus_pc@asus:/mnt/c/Users/asus pc/OneDrive/Documents/05semester4/IF2224 Teori Bahasa Formal dan Otomata/Tubes/MDK-Tubes-IF2224-2026$ make run
Masukkan File input (contoh: test/input2.txt): test/milestone-1/input-6.txt
Menyimpan output ke test/output_6.txt
beginsy
comment ( Komentar format pertama )
ident (x)
becomes
intcon (1)
semicolon

comment ( Komentar format kedua )
endsy
period
```

7. **Kasus Uji 7: Struktur Perulangan Lengkap**
   Tujuan: Menguji pengenalan rentetan keyword perulangan (looping) yang berurutan seperti for, to, do, dan while.

Input:

```
test > milestone-1 > input-7.txt
1  begin
2    for i := 1 to 10 do
3      x := x + 1;
4    while x < 5 do
5      x := x + 2;
6  end.
```

Output:

```
asus_pc@asus:/mnt/c/Users/asus pc/OneDrive/Documents/05semester4/IF2224 Teori Bahasa Formal dan Otomata/Tubes/MDK-Tubes-IF2224-2026$ make run
Masukkan File input (contoh: test/input2.txt): test/milestone-1/input-7.txt
Menyimpan output ke test/output_7.txt
beginsy
forsy
ident (i)
becomes
intcon (1)
tosy
intcon (10)
dosy
ident (x)
becomes
ident (x)
plus
intcon (1)
semicolon

whilesy
ident (x)
lss
intcon (5)
dosy
ident (x)
becomes
ident (x)
plus
intcon (2)
semicolon

endsy
period
```

8. **Kasus Uji 8: Deklarasi Array dan Penggunaan Kurung Siku**
   Tujuan: Menguji keyword struktur data kompleks dan pembacaan tanda baca kurung siku pada saat deklarasi maupun akses elemen.

Input:

```
test > milestone-1 > input-8.txt
var
  arr: array [5] of integer;
begin
  arr[1] := 100;
end.
```

Output:

```
asus_pc@asus:/mnt/c/Users/asus pc/OneDrive/Documents/05semester4/IF2224 Teori Bahasa Formal dan Otomata/Tubes/WOK-Tubes-IF2224-2026$ make run
Masukkan File input (contoh: test/input2.txt): test/milestone-1/input-8.txt
Menyimpan output ke test/output_8.txt
varsy
ident (arr)
colon
arraysy
lbrack
intcon (5)
rbrack
ofsy
ident (integer)
semicolon

beginsy
ident (arr)
lbrack
intcon (1)
rbrack
becomes
intcon (100)
semicolon

endsy
period
```

9. **Kasus Uji 9: Identifier Alfanumerik Sangat Panjang**
   Tujuan: Menguji keandalan DFA dalam menerima string identifier panjang yang merupakan kombinasi huruf dan angka tanpa terpotong.

Input:

```
test > milestone-1 > input-9.txt
var
  identiniSangatPanjang24tetapiTetapValid: integer;
begin
  identiniSangatPanjang24tetapiTetapValid := 100;
end.
```

Output:

```
asus_pc@asus:/mnt/c/Users/asus pc/OneDrive/Documents/05semester4/IF2224 Teori Bahasa Formal dan Otomata/Tubes/WOK-Tubes-IF2224-2026$ make run
Masukkan File input (contoh: test/input2.txt): test/milestone-1/input-9.txt
Menyimpan output ke test/output_9.txt
varsy
ident (identiniSangatPanjang24tetapiTetapValid)
colon
ident (integer)
semicolon

beginsy
ident (identiniSangatPanjang24tetapiTetapValid)
becomes
intcon (100)
semicolon

endsy
period
```

10. **Kasus Uji 10: Stress Test (Whitespace dan Newlines Berlebih)**
    Tujuan: Menguji ketepatan fungsi pengabaian spasi kosong, tab, dan baris baru (enter) secara berlebihan pada lexer di dalam sebuah blok program.

Input:

```
test > milestone-1 > input-10.txt
begin
    x       :=          10   ;


end.
```

Output:

```
Masukkan File input (contoh: test/input2.txt): test/milestone-1/input-10.txt
Menyimpan output ke test/output_10.txt
beginsy
ident (x)
becomes
intcon (10)
semicolon

endsy
period
```

4 Kesimpulan dan Saran

4.1 Kesimpulan

Berdasarkan hasil perancangan, implementasi, dan pengujian yang telah dilakukan, dapat disimpulkan bahwa program *lexer* untuk bahasa pemrograman "Arion" telah berhasil dibangun dan memenuhi seluruh spesifikasi yang diminta. Penggunaan arsitektur *modular* berbasis pemrograman berorientasi objek pada C++ secara akurat mampu menerjemahkan aturan Diagram Transisi DFA ke dalam bentuk pergantian

state numerik secara eksplisit tanpa bergantung pada pustaka *Regular Expression* eksternal.

Hasil dari berbagai kasus uji membuktikan bahwa *lexer* beroperasi dengan konsisten dalam mengidentifikasi seluruh token, mulai dari *keywords*, *identifiers*, literal numerik (*integer* dan riil), literal *string*, operator, hingga *delimiter*. Selain itu, *lexer* telah dengan sukses menjalankan peran utamanya untuk mengabaikan karakter yang tidak relevan untuk dieksekusi, yakni spasi kosong (*whitespace*), baris baru (*newline*), serta format komentar { } dan `(* *)`.

4.2 Saran

Untuk memfasilitasi integrasi kode pada tahap selanjutnya, yaitu Analisis Sintaks (*Syntax Analysis* / Milestone 2), terdapat beberapa penyesuaian yang disarankan untuk diterapkan pada *lexer*:

1. **Penambahan Pelacakan Posisi (Baris dan Kolom):** Saat ini *lexer* berfokus pada pengembalian jenis dan nilai token. Ke depannya, struktur data token perlu direfaktor agar turut menyimpan rekam jejak nomor baris (*line*) dan letak kolom (*column*). Hal ini sangat krusial agar *parser* dapat menghasilkan pesan *error* sintaksis yang presisi dan informatif bagi pengguna.

2. **Implementasi Pemanggilan *On-Demand* (*Lazy Evaluation*):** Saat ini program langsung mengevaluasi seluruh berkas secara utuh di awal (*analyze-all*). Untuk kebutuhan pemrosesan *parser* yang optimal, modul *lexer* sebaiknya menyediakan antarmuka pemanggilan seperti `getNextToken()`. Fungsi ini memungkinkan *parser* meminta pasokan token satu per satu hanya ketika dibutuhkan saat melakukan evaluasi pohon sintaksis.

3. **Abstraksi Objek Token (*Token Stream*):** Keluaran dari proses leksikal sebaiknya tidak hanya dicetak langsung ke layar atau berkas teks, melainkan dirangkai ke dalam bentuk struktur data list dari objek `Token` di dalam memori program agar struktur abstraknya mudah ditelusuri ulang oleh program *parser*.

5 Lampiran

- **Link Repository GitHub:**
  `https://github.com/Zannn210/WOK-Tubes-IF2224-2026`

- **Link Workspace Diagram Transisi DFA:**
  `https://app.diagrams.net/#G1-bUTazvM-vi0PhCwLhb2eJne1TykH-1k#%7B%22pageId%22%3A%22ddC6T3cmnvWE-0hMUWC9%22%7D`

- **Tabel pembagian tugas:**

Nama Lengkap	**NIM**	**Tugas**	**Kontribusi**
Fauzan Mohamad Abdul Ghani	13524113	Core System dan Repository Manager: Menginisialisasi repository GitHub dan struktur folder utama, membuat skrip Makefile, membangun kerangka dasar Lexer untuk menerima input file `.txt` dan menghasilkan daftar token, serta membuat penanganan error.	20%
Muhammad Daffa Arrizki Yanma	13524133	DFA Implementer dan Designer (Keywords dan Identifiers): Merancang dan menggambar diagram transisi DFA secara manual menggunakan workspace web untuk kategori keywords dan identifier, serta mengimplementasikannya dalam C/C++.	20%
Eduard Daniel Ariajaya	13524129	DFA Implementer dan Designer (Numbers, Strings, dan Comments): Merancang diagram transisi DFA untuk konstanta numerik, string, dan comment multi-baris, serta mengimplementasikan logikanya ke dalam modul C/C++.	20%
Safira Berlianti	13524128	DFA Implementer dan Designer (Operators dan Delimiters): Merancang diagram transisi DFA untuk operator dan delimiter, mengintegrasikan seluruh bagian DFA, serta mengimplementasikannya dalam C/C++.	20%
Reysha Syafitri	13524137	Quality Assurance dan Technical Writer: Membuat minimal 5 kasus uji unik beserta file input/output, menulis dokumentasi `README.md`, menyusun laporan PDF, serta memastikan konfigurasi release GitHub sesuai format tag.	20%

Table 1: Pembagian tugas anggota tim

6 Referensi

1. *Compiler Design - Lexical Analysis*. Diakses dari: `https://www.tutorialspoint.com/compiler_design/compiler_design_lexical_analysis.htm`
2. Hopcroft, J. E., Motwani, R., & Ullman, J. D. *Introduction to Automata Theory, Languages, and Computation* (3rd Edition).
3. Aho, A. V., Lam, M. S., Sethi, R., & Ullman, J. D. (2006). *Compilers: Principles, Techniques, and Tools*.
4. *An Introduction to Makefiles*. GNU. Diakses dari: `https://www.gnu.org/software/make/manual/html_node/Introduction.html`